



Énoncé – Projets IA

Ce document contient **deux énoncés de projets réels**, simples mais professionnels, chacun **réalisable en ~4 heures**, intégrant **LangChain, RAG, Vector DB et Agents / LangGraph**.

◆ PROJET 1 : Assistant IA d'analyse de CV (RH)

1. Contexte

Dans un contexte de recrutement, les recruteurs reçoivent un grand nombre de CV sous format PDF. L'analyse manuelle de ces CV est chronophage et répétitive.

L'objectif de ce projet est de développer un **assistant IA RH** capable d'analyser automatiquement un CV et d'aider le recruteur à :

- comprendre rapidement le profil du candidat
 - poser des questions sur le CV
 - évaluer l'adéquation avec une offre d'emploi
-

2. Objectif pédagogique

À la fin de ce projet, l'apprenant sera capable de :

- implémenter une architecture **RAG simple**
 - utiliser une **Vector Database** pour la recherche sémantique
 - créer un **agent IA basique** pour choisir une action
 - intégrer LangChain dans un cas réel (RH)
-

3. Fonctionnalités attendues

L'application doit permettre :

1. L'upload d'un CV (PDF)
2. L'extraction et le découpage du texte

3. La création d'embeddings et leur stockage dans une Vector DB
 4. La possibilité pour l'utilisateur de :
 - poser une question sur le CV
 - demander un résumé du profil
 - demander un score de compatibilité avec une offre
-

4. Périmètre technique

- LangChain
 - Vector DB : ChromaDB ou FAISS
 - LLM (OpenAI / Mistral / Llama)
 - Python
 - LangGraph (agent simple)
 - Streamlit (interface)
-

7. Étapes de réalisation (≈ 4h)

1. Mise en place de l'environnement
 2. Parsing et découpage du CV
 3. Implémentation du RAG
 4. Création de l'agent décisionnel
 5. Tests et validation avec langsmith
 6. création d'interface
-

◆ PROJET 2 : Agent IA pour automatiser des tâches internes

1. Contexte

Dans de nombreuses entreprises, les employés effectuent quotidiennement des tâches répétitives telles que :

- résumer des documents
- rédiger des emails
- analyser une demande utilisateur

Ce projet vise à créer un **agent IA polyvalent** capable d'automatiser ces tâches.

3. Fonctionnalités attendues

L'agent doit être capable de :

1. Identifier le type de demande utilisateur
 2. Choisir automatiquement l'outil approprié
 3. Exécuter l'action et retourner le résultat
-

5. Stack technique

- LangChain
 - LangGraph
 - LLM
 - Python
-

6. Description du graphe LangGraph

Le graphe doit contenir :

- Un nœud de décision
- Un nœud par outil
- Un état final

Le graphe peut être simple et linéaire.

7. Étapes de réalisation (≈ 4h)

1. Création des outils Python
 2. Création de l'agent LangChain
 3. Implémentation du graphe LangGraph
 4. Tests de scénarios réels
 5. création streamlit
-